

Índice

- 1.- Operadores Aritméticos
- 2.- Operadores Relacionales
- 3.- Operadores Lógicos
- 4.- Asignaciones especiales
- 5.- Operadores sufijo y postfijo
- 6.- Prioridad de los operadores
- 7.- Instrucciones de control. ALTERNATIVAS
 - 7.1.- Alternativa simple
 - 7.2.- Alternativa doble
 - 7.3.- Alternativa múltiple
- 8.- Instrucciones de control. REPETITIVAS
 - 8.1.- Mientras
 - 8.2.- Repetir
 - 8.3.- Para
- 9.1.- Ejemplo de módulo interno
- 9.2.- Ejemplo de módulo externo
- 10.1.- Creación de la clase yeti
- 10.2.- Programa de prueba de la clase yeti
- 11.1- Ejemplo 2
- 11.2.- Prueba ejemplo 2
- 12.1.- Ejemplo 3
- 12.2.- Prueba ejemplo 3
- 13.1.- Ejemplo 4
- 13.2.- Prueba ejemplo 4
- 13.- Clase o instancia
- 14.- Uso de String (cadenas)

- 15.- Wrappers, object y string
- 15.1.- Método equals
- 15.2.- Método toString
- 16.- Estructuras estáticas

* UML

JAVA

1.- OPERADORES ARITMÉTICOS

+	suma
-	resta
*	multiplicación
/	división
%	resto

+	concatenación
---	---------------

2.- OPERADORES RELACIONALES

==	igual
!=	distinto
<	menor
>	mayor
<=	menor o igual
>=	mayor o igual

3.- OPERADORES LÓGICOS

&&	true si op1 y op2 son true. No necesita evaluar op2 si op1 es false
&	true si op1 y op2 son true. Simple evalúa op2
	true si op1 u op2 son true. No necesita evaluar op2 si op1 es true
	true si op1 u op2 son true. Simple evalúa op2
^	true si op1 y op2 tienen distinto valor
!	true si op es false y false si op es true

4.- ASIGNACIONES ESPECIALES

x+=y	x = x+y
x-=y	x = x-y
x*=y	x = x*y
x/=y	x = x/y
x%=y	x = x%y

5.- OPERADORES DE SUFIJO Y POSTFIJO

++	--
----	----

6.- PRIORIDAD DE LOS OPERADORES

```
Operadores postfijos expr++ expr--
Operadores unarios ++expr --expr +expr -expr !
Creación o cast new (type)expr
* / %
+ -
relacionales < > <= >=
igualdad == !=
&
^
|
&&
||
condicional ?:
asignación = += -= /= %=
```

7.- INSTRUCCIONES DE CONTROL. ALTERNATIVAS

7.1- ALTERNATIVA SIMPLE

```
if (<expresiónBooleana>){
    //Bloque instrucciones
} //Fin Si
```

7.2.- ALTERNATIVA DOBLE

```
if (<condición>){
    //Bloque instrucciones
}else{
    //Bloque instrucciones
} //Fin Si
```

7.3.- ALTERNATIVA MÚLTIPLE

```
if(expresiónBooleana){
    //Bloque instrucciones
}else if{
    //Bloque instrucciones
}else if{
    //Bloque instrucciones
}else{
    //Bloque instrucciones
} //Fin Si
```

8.- INSTRUCCIONES DE CONTROL. REPETITIVAS

8.1.- MIENTRAS

```
while(<condición>{
    //Bloque instrucciones
} //Fin Mientras
```

8.1.- REPETIR

```
do{
    //Bloque instrucciones
}while (<expresiónBooleana>);
```

8.2.- PARA

```
for (inicialización; exprBooleana; incr){
    //Bloque instrucciones
} //Fin Para
```

9.1.- EJEMPLO MÓDULO INTERNO

```
public class PruebaModulos {  
  
    // 1. AQUÍ AÑADES EL MÓDULO (Procedimiento)  
    public static void pintaSuma(double x, double y) { // x  
e y son parámetros formales  
        double suma = x + y; // Variable local  
        System.out.println("La suma es: " + suma);  
    }  
  
    // Programa principal  
    public static void main(String[] args) {  
        double a = 3.0;  
        double b = 5.0;  
  
        // 2. LLAMADA AL MÓDULO  
        pintaSuma(a, b); // a y b son los parámetros  
actuales  
    }  
}
```

9.2.- EJEMPLO MÓDULO EXTERNO

```
//fichero calculadora.java  
public class Calculadora {  
    // Este es tu módulo externo  
    public static void pintaSuma(double x, double y) {  
        double suma = x + y;  
        System.out.println("La suma es: " + suma);  
    }  
}  
  
//fichero principal.java  
public class Principal {  
    public static void main(String[] args) {  
        double a = 10;  
        double b = 20;  
  
        // LLAMADA AL MÓDULO EXTERNO  
        // Sintaxis: Clase.modulo(parámetros);  
        Calculadora.pintaSuma(a, b);  
    }  
}
```

10.1.- CREACIÓN DE CLASE YETI

```
public class Yeti {  
    // Atributos  
    String color;  
    char sexo;  
  
    // 1. Constructor por defecto (sin parámetros)  
    public Yeti() {  
        this.color = "Blanco";  
        this.sexo = 'M';  
    }  
  
    // 2. Constructor parametrizado (basado en  
    características concretas)  
    public Yeti(String color, char sexo) {  
        this.color = color;  
        this.sexo = sexo;  
    }  
  
    // 3. Constructor copia (crea un Yeti a partir de otro  
    existente)  
    public Yeti(Yeti otroYeti) {  
        this.color = otroYeti.color;  
        this.sexo = otroYeti.sexo;  
    }  
  
    // Sobreescribimos toString para que devuelva una  
    cadena que represente al objeto  
    @Override  
    public String toString() {  
        return "Yeti [Color: " + color + ", Sexo: " + sexo + "];"  
    }  
}
```

10.2.- PROGRAMA DE PRUEBA DE CLASE YETI

```
public class Principal {  
    public static void main(String[] args) {  
        Yeti yeti1 = new Yeti(); // Usa constructor por defecto  
        Yeti yeti2 = new Yeti("Marrón", 'F'); // Usa constructor  
        parametrizado  
        Yeti yetiCopia = new Yeti(yeti2); // Usa constructor copia  
  
        System.out.println(yeti1.toString()); // Yeti [Color: Blanco,  
        Sexo: M]  
        System.out.println(yetiCopia.toString()); // Yeti [Color:  
        Marrón, Sexo: F]  
    }  
}
```

11.1.- EJEMPLO 2

```
public class Producto {
    // 1. Atributos (Estado - Privados por encapsulamiento)
    private String nombre;
    private double precio;
    private int stock;

    // 2. Constructores (Para inicializar el objeto)
    // Constructor vacío
    public Producto() {
    }

    // Constructor con parámetros
    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    // 3. Getters y Setters (Comportamiento - Para leer/modificar atributos)
    // Getter para leer el precio
    public double getPrecio() {
        return this.precio;
    }

    // Setter para modificar el precio
    public void setPrecio(double nuevoPrecio) {
        if (nuevoPrecio >= 0) { // ¡Validación de seguridad!
            this.precio = nuevoPrecio;
        } else {
            System.out.println("Error: El precio no puede ser negativo.");
        }
    }

    // (Aquí irían los getters y setters del resto de atributos...)
}
```

11.2.- PRUEBA EJEMPLO 2

```
public class TiendaMain {
    public static void main(String[] args) {

        // Forma 1: Usando el constructor vacío y luego los Setters
        Producto laptop = new Producto();

        // Aquí respondemos a la pregunta anterior: usamos el setter para darle el precio
        laptop.setPrecio(800.50);
        // (Imagina que también usamos
        laptop.setNombre("Portátil");)

        // Forma 2: Usando el constructor con parámetros (¡más rápido!)
        Producto raton = new Producto("Ratón Inalámbrico", 25.99, 50);

        // Intentamos poner un precio inválido para ver actuar a nuestro "guardián"
        raton.setPrecio(-5.0);
    }
}
```

12.1.- EJEMPLO 3

```
public class Producto {
    // 1. ATRIBUTOS (Estado privado)
    private String nombre;
    private double precio;
    private int stock;

    // 2. CONSTRUCTORES (Para crear el objeto)
    public Producto() {
        // Constructor vacío
    }

    public Producto(String nombre, double precio, int
stock) {
        // Constructor con parámetros
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    // 3. GETTERS Y SETTERS (Para leer/modificar de
forma segura)
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return this.nombre;
    }

    public void setPrecio(double nuevoPrecio) {
        if (nuevoPrecio >= 0) {
            this.precio = nuevoPrecio;
        } else {
```

```
            System.out.println("Error: El precio no puede ser
negativo.");
        }
    }

    public double getPrecio() {
        return this.precio;
    }

    // 4. MÉTODOS PERSONALIZADOS
(Comportamiento extra)
    public int contarMayusculas() {
        int contador = 0;
        if (this.nombre != null) {
            for (int i = 0; i < this.nombre.length(); i++) {
                if
(Character.isUpperCase(this.nombre.charAt(i))) {
                    contador++;
                }
            }
        }
        return contador;
    }

    public String obtenerPrecioOculto() {
        String precioTexto = String.valueOf(this.precio);
        return precioTexto.replaceAll("[0-9]", "*");
    }
}
```

12.2.- PRUEBA EJEMPLO 3

```
public class ProgramaTienda {
    public static void main(String[] args) {

        // 1. Creamos un producto usando el constructor
        // con parámetros
        Producto laptop = new Producto("Portátil GAMER",
        850.75, 10);

        // 2. Usamos los métodos personalizados que
        // creamos
        System.out.println("Analizando el producto: " +
        laptop.getNombre());

        int totalMayusculas = laptop.contarMayusculas();
        System.out.println("Cantidad de letras
        mayúsculas: " + totalMayusculas);

        System.out.println("El precio oculto es: " +
        laptop.obtenerPrecioOculto());

        // 3. Probamos la seguridad de nuestro Setter
        System.out.println("Intentando poner un precio
        negativo...");
        laptop.setPrecio(-50.0); // Esto lanzará nuestro
        mensaje de error
    }
}
```

13.- CLASE O INSTANCIA

¿Este dato es único para cada objeto o es algo que comparten todos por igual?

- Si el atributo es el DNI de una persona -> **Instancia** (cada persona tiene el suyo).
- Si el atributo es la **Población Total** de personas -> **Clase** (es un contador común).
- Si el método es **caminar** -> **Instancia** (cada persona camina por su cuenta).
- Si el método es **convertirEurosADolares** -> **Clase** (es una herramienta matemática que no depende de quién sea la persona).

```
public class Coche {
    // ATRIBUTO DE INSTANCIA: Cada coche tiene su propio color
    private String color;

    // ATRIBUTO DE CLASE: Todos los coches salen de la misma
    // marca
    private static String marca = "Toyota";

    // MÉTODO DE INSTANCIA: El coche específico acelera
    public void acelerar() {
        System.out.println("El coche " + this.color + " está acelerando.");
    }

    // MÉTODO DE CLASE: Es una utilidad general de la fábrica
    public static void mostrarGarantia() {
        System.out.println("La garantía es de 5 años para todos los " +
        marca);
    }
}
```

14.- USO DE STRING (CADENAS)

```
public class ManejoCadenas {
    public static void main(String[] args) {
        String texto = " Aprender Java es genial ";

        // 1. Quitar espacios a los lados e imprimir longitud
        texto = texto.trim();
        System.out.println("Longitud: " + texto.length());

        // 2. Extraer una parte (desde el índice 0 hasta el 8
        // excluido)
        String primeraPalabra = texto.substring(0, 8);
        System.out.println("Palabra: " + primeraPalabra); //
        Imprime "Aprender"

        // 3. Reemplazar caracteres
        String modificado = texto.replace('a', '@');
        System.out.println("Reemplazo: " + modificado);

        // 4. Comparación correcta
        String clave1 = "hola";
        String clave2 = new String("hola");

        System.out.println("Comparar memoria (==): " + (clave1 ==
        clave2)); // false
        System.out.println("Comparar contenido (equals): " +
        clave1.equals(clave2)); // true
    }
}
```

15.- WRAPPERS, OBJECT Y STRING

Las clases Wrappers son envoltorios para los tipos primitivos que permiten convertir los primitivos a objetos y viceversa.

API: java.lang

En esa API tenemos:

- **Métodos de instancia para convertir tipos a primitivos:** intValue, floatValue...
- **Un método estático para convertir una cadena en un tipo específico** (comienza con parse)
- **Métodos de nombre toString** que convierten el objeto a cadena.
- **Varios métodos para transformar un tipo en otro**

15.1.- MÉTODO EQUALS

Este método devuelve verdadero si los dos objetos son iguales

boolean	equals(Object obj)
---------	--------------------

En Object dos objetos son iguales si son el mismo (si tienen la misma dirección de memoria)

Ejemplo: x.equals(y)

15.2.- MÉTODO TOSTRING

Devuelve una cadena que represente al objeto.

String	toString()
--------	------------

15.3.- CLASE STRING

Tenemos dos formas de usar cadenas de caracteres:

- Como un conjunto de datos char
- Utilizando un String

String se puede usar de dos formas distintas:

- String str1,str2;
- str1="abc";
str2=new String("abc");

Si lo hacemos de la **primera forma**, al comparar las dos cadenas con `==` y con `equals` nos dará que de las dos formas es **true**.

Si lo hacemos de la **segunda forma**, al comparar las dos cadenas con `==` siempre dará **false** mientras que si lo comparamos con `equals` siempre dará **true**.

16.- ESTRUCTURAS ESTÁTICAS

El uso de estructuras nos facilitan el manejo de los datos, considerándose de forma general los siguientes datos:

- **Internos**: residen en la memoria principal.
- **externos**: residen en una memoria auxiliar.
- **estáticos**: su tamaño se define en la creación de la estructura y no se puede modificar durante el uso de la misma.
- **dinámicos**: su tamaño se puede modificar durante el uso de la estructura.
- **compuestos**: los registros y estructuras son datos internos que se forman por un conjunto de tipos básicos y derivados.
- **lineales**: pueden estar enlazados sólo con un elemento anterior y otro posterior.
- **no lineales**: pueden enlazarse con más de un elemento anterior y más de uno posterior.

17.- HERENCIAS

17.1.- SUPERCLASE

```
public class Producto {
    protected String nombre; // Usamos 'protected' para
    que los hijos puedan verlo
    protected double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public void mostrarInfo() {
        System.out.println("Producto: " + nombre + " |
        Precio: " + precio);
    }
}
```

17.2.- SUBCLASE Y EXTENDS

```
// 'Electronico' hereda todo lo de 'Producto'
public class Electronico extends Producto {
    private int mesesGarantia;

    public Electronico(String nombre, double precio, int
    mesesGarantia) {
        super(nombre, precio); // Aquí le pasamos los
        datos al constructor de Producto
        this.mesesGarantia = mesesGarantia; // Luego
        inicializamos lo que es único del hijo
    }
}
```

18.- SOBRESCRITURA DE METODOS (@Override)

Clase Producto

```
public void mostrarInfo() {
    System.out.println("Producto: " + nombre + " | Precio: "
    + precio);
}
```

Como implementarlo para que muestre toda info de electronico:

```
@Override
public void mostrarInfo() {
    super.mostrarInfo(); // Llamamos a la versión del
    padre (imprime nombre y precio)
    System.out.println("Meses de garantia: " +
    mesesGarantia); // Añadimos el dato extra
}
```

Ejemplo:

```
Electronico raton = new Electronico("Raton", 19.20, "12");
raton.mostrarInfo();
```

Otro ejemplo:

```
// --- ARCHIVO: Producto.java ---
public class Producto {
    // Usamos 'protected' para que las subclases
    accedan directamente
    protected String nombre;
    protected double precio;

    // Constructor del padre
    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public void mostrarInfo() {
        System.out.println(" Producto: " + nombre + " |
Precio: " + precio + "€");
    }
}

// --- ARCHIVO: Electronico.java ---
public class Electronico extends Producto {
    private int mesesGarantia;

    public Electronico(String nombre, double precio, int
mesesGarantia) {
        super(nombre, precio); // Llama al constructor de
Producto
        this.mesesGarantia = mesesGarantia;
    }

    @Override
    public void mostrarInfo() {
        super.mostrarInfo(); // Reutiliza el código del
```

```
padre
        System.out.println(" Garantía: " + mesesGarantia + "
meses");
    }
}

// --- ARCHIVO: Alimento.java ---
public class Alimento extends Producto {
    private String fechaCaducidad;

    public Alimento(String nombre, double precio, String
fechaCaducidad) {
        super(nombre, precio); // Llama al constructor de
Producto
        this.fechaCaducidad = fechaCaducidad;
    }

    @Override
    public void mostrarInfo() {
        super.mostrarInfo(); // Reutiliza el código del
padre
        System.out.println(" Caducidad: " +
fechaCaducidad);
    }
}
```

Programa que lo utiliza:

```
public class TiendaPrincipal {
    public static void main(String[] args) {

        // 1. Creamos objetos de las subclases
        // Aunque son de tipos diferentes, ambos "son"
        // productos.
        Electronico miLaptop = new Electronico("Laptop
        Pro", 1200.0, 24);
        Alimento miManzana = new Alimento("Manzana
        Fuji", 0.50, "20/12/2024");

        // 2. Los metemos en una lista de la clase Padre
        // Java nos permite esto porque un Alimento "es
        // un" Producto.
        Producto[] listaProductos = {miLaptop,
        miManzana};

        System.out.println("--- DETALLE DEL INVENTARIO
        ---");

        // 3. Recorremos la lista y llamamos a mostrarInfo()
        for (Producto p : listaProductos) {
            // Aquí ocurre la magia: Java sabe qué
            // mostrarInfo() ejecutar
            // (el del electrónico o el del alimento)
            // automáticamente.
            p.mostrarInfo();
            System.out.println("-----");
        }
    }
}
```

¿Qué es una Clase Abstracta?

Imagina que la clase Producto es ahora una *idea* o un **concepto**, no algo que puedas tocar. En Java, al marcarla como **abstract**, le decimos al compilador: *"No permitas que nadie haga new Producto(), porque un producto genérico no existe en mi tienda"*.

Además, podemos crear **métodos abstractos**. Son métodos que el padre anuncia pero no escribe, obligando a cada hijo a terminarlos.

Vamos a añadir un método abstracto llamado `calcularPrecioFinal()`. Como los alimentos y la electrónica tienen impuestos (IVA) diferentes, el padre no sabe cómo calcularlo, así que le pasa la responsabilidad a los hijos.

Clase padre abstracta:

```
// Añadimos la palabra 'abstract'
public abstract class Producto {
    protected String nombre;
    protected double precioBase;

    public Producto(String nombre, double precioBase) {
        this.nombre = nombre;
        this.precioBase = precioBase;
    }

    // MÉTODO ABSTRACTO: No tiene cuerpo {},
    termina en punto y coma.
    // Obliga a los hijos a implementar su propia lógica
    de IVA.
    public abstract double calcularPrecioFinal();

    public void mostrarInfo() {
        System.out.println("Producto: " + nombre);
    }
}
```

Hijos:

```
public class Alimento extends Producto {
    public Alimento(String nombre, double precioBase) {
        super(nombre, precioBase);
    }
    @Override
    public double calcularPrecioFinal() {
        return this.precioBase * 1.04; // IVA superreducido
        del 4%
    }
}
```

```
public class Electronico extends Producto {
    public Electronico(String nombre, double
    precioBase) {
        super(nombre, precioBase);
    }

    @Override
    public double calcularPrecioFinal() {
        return this.precioBase * 1.21; // IVA general del 21%
    }
}
```

Otro ejemplo:

```
// 1. Definición de la clase abstracta
// No se pueden crear objetos con 'new Producto()'
public abstract class Producto {
    // 2. Atributos protegidos: los hijos pueden verlos directamente
    protected String nombre;
    protected double precioBase;

    // 3. Constructor: Aunque la clase es abstracta, tiene constructor
    // para que los hijos lo llamen mediante super()
    public Producto(String nombre, double precioBase) {
        this.nombre = nombre;
        this.precioBase = precioBase;
    }

    // 4. Método concreto: Comportamiento compartido por todos los hijos
    public void mostrarNombre() {
        System.out.println(" Artículo: " + nombre);
    }

    // 5. MÉTODO ABSTRACTO: El "contrato"
    // No tiene cuerpo {}. Obliga a los hijos a definir su propio IVA.
    public abstract double calcularPrecioFinal();
}

// 6. Subclase concreta en otro archivo: Implementa el contrato
public class Alimento extends Producto {
    private String fecha;
```

```
    public Alimento(String nombre, double precioBase, String fecha) {
        super(nombre, precioBase); // Llama al constructor del padre
        this.fecha = fecha;
    }

    @Override
    public double calcularPrecioFinal() {
        return precioBase * 1.04; // IVA del 4% para alimentos
    }
}
```

Programa que lo usa:

```
public class TiendaApp {
    public static void main(String[] args) {
        // Aunque no podemos crear un 'Producto', podemos usarlo como TIPO
        Producto miCompra;

        // Instanciamos un hijo
        miCompra = new Alimento("Pan Artesanal", 2.50, "15/05/2026");

        // Usamos métodos del padre y del hijo
        miCompra.mostrarNombre();
        System.out.println("Precio con IVA: " + miCompra.calcularPrecioFinal() + "€");
    }
}
```

INTERFACES

Interfaz:

```
public interface Enviable {  
    // Todos los métodos en una interfaz son abstractos  
    // por defecto  
    void calcularCostoEnvio(double distancia);  
    void generarEtiqueta();  
}
```

Clase:

```
public class Electronico extends Producto implements  
Enviable {  
  
    public Electronico(String nombre, double  
precioBase) {  
        super(nombre, precioBase);  
    }  
  
    // Obligación 1: El contrato de la Clase Abstracta  
    // Producto  
    @Override  
    public double calcularPrecioFinal() {  
        return precioBase * 1.21;  
    }  
  
    // Obligación 2: El contrato de la Interfaz Enviable  
    @Override  
    public void calcularCostoEnvio(double distancia) {  
        System.out.println("Costo de envío para  
electrónico: " + (distancia * 0.5));  
    }  
  
    @Override  
    public void generarEtiqueta() {  
        System.out.println("Generando etiqueta para: " +
```

```
getNombre());  
    }  
}
```

Programa que prueba:

```
public class PruebaInterfaz {  
    public static void main(String[] args) {  
        // 1. Creamos objetos de distintas clases que  
        // implementan Enviable  
        Electronico laptop = new Electronico("MacBook  
Air", 1100.0);  
        System.out.println("GESTIÓN DE ENVÍOS");  
        // 2. Usamos el objeto con sus métodos de  
        // interfaz  
        laptop.generarEtiqueta();  
        laptop.calcularCostoEnvio(150.5); // Distancia en  
        km  
        System.out.println("-----");  
  
        // 3. El poder del Polimorfismo con Interfaces  
        // Podemos crear una variable del tipo de la  
        // interfaz  
        Enviable paquete;  
        paquete = laptop; // Esto es válido porque  
        Electronico "es un" Enviable  
        System.out.println("Procesando paquete  
genérico...");  
        paquete.generarEtiqueta();  
    }  
}
```

Interfaz multiple

```
public interface Reinicialable {
    void reiniciarAFabrica();
}
```

```
public class Smartphone extends Producto
implements Enviabile, Reinicialable {

    public Smartphone(String nombre, double
precioBase) {
        super(nombre, precioBase);
    }
    // Cumplimos con el contrato de Producto (Clase
Abstracta)
    @Override
    public double calcularPrecioFinal() {
        return precioBase * 1.21;
    }
    // Cumplimos con el contrato de Enviabile (Interfaz 1)
    @Override
    public void calcularCostoEnvio(double distancia) {
        System.out.println("Costo de envío: 5€ tarifa
plana.");
    }
    @Override
    public void generarEtiqueta() {
        System.out.println("Etiqueta para Smartphone
lista.");
    }
    // Cumplimos con el contrato de Reinicialable
(Interfaz 2)
    @Override
    public void reiniciarAFabrica() {
        System.out.println("Borrando datos y restaurando
```

```
sistema...");
    }
}
```

```
public class PruebaMultiplesInterfaces {
    public static void main(String[] args) {
        // 1. Creamos el objeto concreto
        Smartphone miS24 = new Smartphone("Samsung
S24", 900.0);
        System.out.println("PRUEBA INTERFACES
MULTIPLES");
        // --- Identidad como PRODUCTO ---
        // Accedemos a lo que heredó de la clase
abstracta
        System.out.println("Nombre: " +
miS24.getNombre());
        System.out.println("Precio con IVA: " +
miS24.calcularPrecioFinal() + "€");
        // --- Identidad como ENVIABLE ---
        // Tratamos al smartphone solo como algo que se
puede enviar
        Enviabile paquete = miS24;
        paquete.generarEtiqueta();
        paquete.calcularCostoEnvio(20.0);
        // --- Identidad como REINICIABLE ---
        // Aquí lo tratamos como un dispositivo que
requiere mantenimiento
        Reinicialable dispositivo = miS24;
        dispositivo.reiniciarAFabrica();
    }
}
```

Como crear Arrays

```
// Opción A: Crear el array vacío indicando el tamaño
(ej: 5 espacios)
String[] nombres = new String[5];
// Opción B: Crear el array y llenarlo de golpe (Java
calcula el tamaño solo)
int[] numeros = {10, 20, 30, 40};
```

Cómo utilizarlos

```
public class EjemploArray {
    public static void main(String[] args) {
        String[] nombres = new String[3];
        // Array de 3 huecos (0, 1 y 2)
        // 1. Asignar valores
        nombres[0] = "Ana";
        nombres[1] = "Luis";
        nombres[2] = "Carlos";
        //nombres[3] = "Marta"; //¡ERROR! Esto haría
        explotar el programa porque el hueco 3 no existe.
        // 2. Leer un valor
        System.out.println("El primer nombre es: " +
        nombres[0]);
        // 3. Recorrer todos los valores con un bucle
        System.out.println("Lista completa:");
        for (int i = 0; i < nombres.length; i++) {
            System.out.println("Posición " + i + ": " +
            nombres[i]);
        }
    }
}
```

Matrices

```
// Una matriz de 3 filas y 3 columnas (como un tres en
raya)
int[][] tablero = new int[3][3];

// Asignar un valor en la fila 0, columna 2
tablero[0][2] = 1;

// Crearla e inicializarla con datos directamente
int[][] matriz = {
    {1, 2, 3}, // Fila 0
    {4, 5, 6}, // Fila 1
    {7, 8, 9} // Fila 2
};

// Para recorrerla, necesitas un bucle dentro de otro
(bucles anidados)
for (int i = 0; i < matriz.length; i++) { // Recorre filas
    for (int j = 0; j < matriz[i].length; j++) { // Recorre
        columnas
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println(); // Salto de línea al terminar
    cada fila
}
```

La Clase `java.util.Arrays`

`Arrays.sort(tuArray);` Ordena el array de menor a mayor (o alfabéticamente si son textos) en una sola línea.

¡Adiós a programar algoritmos de ordenación complejos!

`Arrays.toString(tuArray);` Convierte el array en un texto bonito como `[Ana, Luis, Carlos]` para poder imprimirlo en consola con un `System.out.println` sin necesidad de hacer un bucle `for`.

`Arrays.equals(array1, array2);` Compara si dos arrays tienen exactamente los mismos elementos en el mismo orden. (Recuerda que usar `array1 == array2` no funciona para comparar su contenido).

Clase con Arrays

```
import java.util.Arrays;

public class AnalizadorTienda {
    /**
     * Demuestra el uso de Arrays Unidimensionales (1D)
     * y los trucos de la clase java.util.Arrays.
     */
    public void gestionarCategorias(String[] categorias) {
        // 1. Convertir el array a un formato de texto legible de golpe
        System.out.println("Orden original de entrada: " + Arrays.toString(categorias));
        // 2. Ordenación automática alfabética usando la utilidad de Java
        Arrays.sort(categorias);
        System.out.println("Ordenado alfabéticamente: " + Arrays.toString(categorias));
        // 3. Uso seguro del último índice válido (longitud - 1)
        int ultimoIndice = categorias.length - 1;
        System.out.println("La última categoría de la lista es: " +
```

```
categorias[ultimoIndice]);
        System.out.println();
    }
    /**
     * Demuestra el uso de Arrays Multidimensionales (Matrices 2D)
     * procesando filas y columnas con bucles anidados.
     */
    public void analizarVentas(double[][] ventas) {
        System.out.println("Estructura: Filas (Tiendas) x Columnas (Trimestres)");
        // El primer bucle recorre cada fila (cada tienda)
        for (int fila = 0; fila < ventas.length; fila++) {
            double totalTienda = 0;
            System.out.print("Tienda #" + fila + " -> Ventas: [ ");

            // El segundo bucle recorre las columnas de esa fila específica
            for (int col = 0; col < ventas[fila].length; col++) {
                System.out.print(ventas[fila][col] + "€ ");
                totalTienda += ventas[fila][col]; // Acumulamos el total
            }

            System.out.println("\n | Total acumulado: " + totalTienda + "€");
        }
        System.out.println();
    }
}
```

Prueba de la clase

```

public class PruebaArrays {
    public static void main(String[] args) {

        // 1. Creamos el objeto analizador
        AnalizadorTienda analizador = new
        AnalizadorTienda();

        // 2. Creamos y probamos el Array Unidimensional
        (1D)
        String[] inventarioCategorias = {"Electrónica",
        "Alimentos", "Ropa", "Juguetes", "Hogar"};

        analizador.gestionarCategorias(inventarioCategorias);

        // 3. Creamos y probamos el Array Bidimensional
        (2D / Matriz)
        // 3 tiendas y sus ventas en 4 trimestres
        double[][] datosVentas = {
            {1200.50, 1500.00, 1800.20, 2100.00}, // Tienda 0
            {950.00, 1100.50, 1050.00, 1300.00}, // Tienda 1
            {2500.00, 2300.00, 2700.80, 3000.50} // Tienda 2
        };
        analizador.analizarVentas(datosVentas);

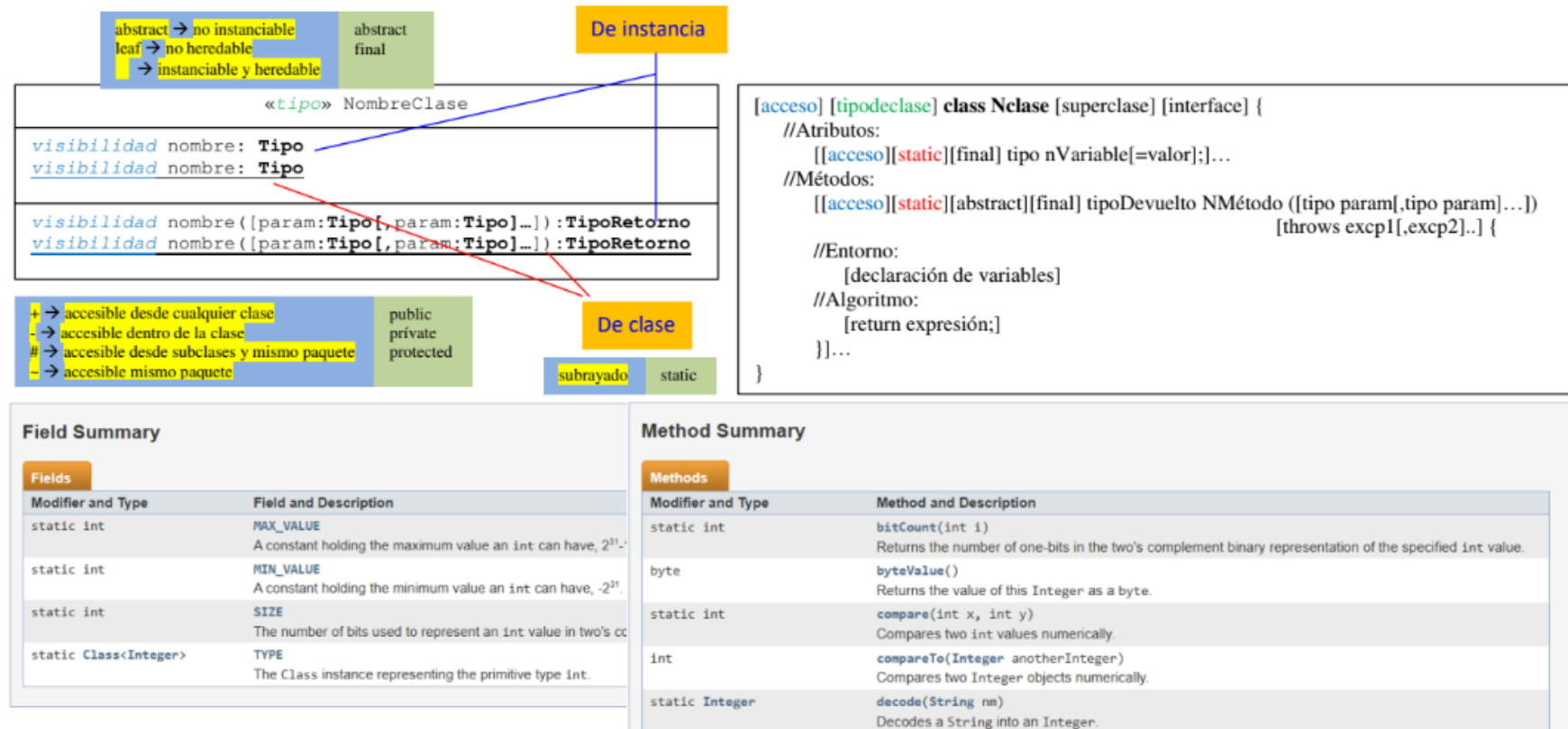
        System.out.println("¡Fin del programa!");
    }
}

```

FALTA

VECTORES UD?
ARRAYS UD?
MATRICES UD?
ESTRUCTURAS DINÁMICAS UD13
CLASES ABSTRACTAS UD?

* UML



Modificador indica acceso (si no hay nada es public pero podría ser protected) y tipo de miembro, de instancia (no aparece nada) o de clase (aparece static)

Tipo indica el tipo de datos que devuelve que podrá ser un tipo primitivo, una clase o void (si es un procedimiento)

Definición, Uso y Documentación

- **Miembro de Instancia (Sin `static`)**

- **Definición (Cómo se escribe):**

- No llevan la palabra reservada `static`.
 - *Ejemplo:* `private int cantidad;`
 - *Ejemplo:* `public int calcularTotal(int sumando) { ... }`

- **Uso en la App (Cómo se llama):**

- Requiere crear el objeto en memoria obligatoriamente con `new`.
 - *Paso 1:* `MiClase obj; //Define variable para el objeto`
 - *Paso 2:* `obj = new MiClase(); //crea objeto usando el constructor por defecto`
 - `obj.cantidad = 10; //Uso del atributo de instancia`
 - `obj.calcularTotal(30); //Uso del método de instancia que recibirá el objeto como mensaje y el parámetro con valor 30, devolviendo 40 (30+10).`

- **Javadoc (Cómo se documenta):**

- Aparecen en las secciones estándar de *Field Summary* o *Method Summary*.
 - La documentación debe enfocarse en cómo el método afecta al **estado particular de ese objeto**.
 - *Etiquetas clave:* `@param` (datos que necesita el objeto), `@return` (qué devuelve ese objeto).

- **Miembro de Clase (Con `static`)**

- **Definición (Cómo se escribe):**

- Llevan obligatoriamente la palabra reservada `static`.
 - *Ejemplo:* `public static int valor;`
 - *Ejemplo:* `public static int sumar(int a, int b) { ... }`

- **Uso en la App (Cómo se llama):**

- No requieren la creación de un objeto. Pertenecen a la clase y la comparten todos los objetos de la misma.
 - Se llama usando directamente el nombre de la clase (Mayúscula inicial).
 - *Ejemplo Atributo:* `NombreClase.atributoEstatico;` (Ej: `Math.PI`)
 - *Ejemplo Método:* `NombreClase.metodoEstatico();` (Ej: `Math.max(5, 10)`)

